

Runtime Security and Threat Mitigation for Autonomous LLM Agents

Complete Project History

Project location: C:\Users\Prajwal\llm-runtime-security

GitHub repository: <https://github.com/PrajwalSxxx/llm-security-gateway.git>

Purpose of this document: This is a chronological engineering record of the work discussed and completed in this conversation. It summarizes the decisions, implementation stages, fixes, demonstrations, tests, documentation, dashboard redesigns, and GitHub updates. It is not a verbatim export of the chat transcript.

1. Original Project Goal

The project goal was to build a controlled local cybersecurity prototype for protecting autonomous LLM agents from indirect prompt injection and unsafe tool execution.

The desired security flow was:

User

```
-> LLM agent
-> proposed tool action
-> Runtime Security Gateway
-> validation
-> intent analysis
-> IPI detection
-> sensitive-data detection
-> risk scoring
-> policy engine
-> Tool Dependency Graph
-> ALLOW / BLOCK / REQUIRE_APPROVAL
-> local tool
```

The project was explicitly required to use fake credentials, local test files, simulated services, local databases, and no real attacks against websites, accounts, financial systems, or third-party infrastructure.

2. Initial Environment Inspection

The machine was inspected before installation.

Environment findings:

- Operating system: Windows 11 Home Single Language.
- Architecture: x64.
- Python: 3.14.2.
- Git: installed.
- Node.js: installed.

- npm: installed.
- Docker: not installed.
- PostgreSQL: not installed.
- Ollama: not installed initially.
- GPU: NVIDIA GeForce RTX 2050.
- GPU memory: approximately 4 GB.
- RAM: approximately 15.73 GB.
- Free storage: approximately 143 to 150 GB during the work.
- Ports 8000, 8501, 5432, and 11434 were initially available.
- VS Code was available.

The project was created under:

C:\Users\Prajwal\llm-runtime-security

3. First Prototype Build

The first implementation created a working local prototype with:

- FastAPI backend.
- Streamlit dashboard.
- Pydantic models.
- SQLite audit database.
- SHA-256 tamper-evident audit chain.
- NetworkX security graph.
- YAML policy configuration.
- Controlled local tools.
- Fake secret files.
- Malicious document examples.
- Local retrieval.
- Pytest tests.
- Dockerfile and Docker Compose files.
- README and beginner documentation.

The initial tools were:

```
read_file()
write_file()
web_search()
database_query()
http_request()
```

The tools were restricted to controlled local resources. No unrestricted shell execution was implemented.

4. Initial Security Features

The first security gateway implemented:

- Tool name validation.
- Basic path validation.
- Protected path detection.
- User intent inference.
- Intent/action mismatch detection.
- Injection keyword signals.
- Sensitive-data regular expressions.
- Risk scores from 0 to 100.
- Policy decisions.
- NetworkX action history.
- ALLOW, BLOCK, and REQUIRE_APPROVAL decisions.
- Audit event storage.
- SHA-256 hash chaining.

Example protected action:

```
{
  "tool": "read_file",
  "arguments": {
    "path": "fake_secrets/api_key.txt"
  },
  "source": "agent"
}
```

This was blocked because the path represented simulated credentials and did not match a normal document-summarization intent.

5. V1 Weaknesses Discovered

The first audit found that the production path was not a real LLM agent.

The original MockAgent:

- Always started with a file-read action.
- Could cause unrelated questions to trigger file operations.
- Used hard-coded string checks to create malicious actions.
- Did not use a local or cloud LLM.
- Used token-overlap retrieval rather than semantic retrieval.

The audit was recorded in:

docs/V1_AUDIT.md

The decision was to preserve the old mock implementation only as legacy/baseline code and replace the production path with Ollama.

6. Ollama Migration

Ollama was installed through the official Windows installer workflow using winget.

Installed version:

ollama 0.34.0

The first installer command timed out while downloading. The installer log was inspected, the installation was completed, and the installed Ollama process was verified.

Ollama was confirmed reachable at:

`http://127.0.0.1:11434`

The application uses the default local endpoint:

`http://localhost:11434`

7. Hardware-Based Model Selection

Because the machine has approximately 16 GB RAM and an RTX 2050 with 4 GB VRAM, a small local model was selected rather than a large model.

Main model:

qwen2.5:3b

Approximate size:

1.9 GB

Embedding model:

nomic-embed-text

Approximate size:

274 MB

Both models were downloaded through Ollama and listed successfully with `ollama list`.

8. Local LLM Verification

The local model was tested before connecting it to the agent.

Test prompts included:

What is the capital of France?

The local model answered Paris.

Explain what an LLM agent is.

The local model produced a meaningful answer.

What is indirect prompt injection?

The local model produced an explanation.

The embedding endpoint was also tested and returned 768-dimensional vectors.

9. Ollama Provider Abstraction

The following files were added:

```
app/llm/base.py
app/llm/ollama_provider.py
app/llm/manager.py
```

The provider supports:

- generate().
- chat().
- structured_output().
- tool_calling().
- embed().
- Model health checks.

The provider refuses non-local endpoints when offline mode is enabled.

If Ollama is unavailable, the system reports:

```
OLLAMA IS NOT RUNNING
Start Ollama and try again.
```

There is no cloud fallback.

10. Real Local Agent

The production agent became RealLocalAgent in app/agent.py.

The agent can return either:

```
final answer
```

or:

```
{
```

```

    "action": "tool",
    "tool": "read_file",
    "arguments": {
        "path": "documents/report.txt"
    }
}

```

The LLM never directly calls Python functions. It only proposes a structured action.

The runtime then sends the action through the security gateway before execution.

11. Local Semantic RAG

The original lexical retrieval was upgraded.

The new local RAG process is:

```

Local document
-> extraction
-> chunking
-> Ollama embedding
-> SQLite vector storage
-> cosine similarity search
-> provenance metadata
-> Ollama response

```

The vector database is local SQLite. No Chroma Cloud, Pinecone, Qdrant Cloud, or hosted embedding service is used.

Each retrieved chunk contains:

```

source
source_type
trust_level
chunk_id
similarity
content

```

Retrieved content is labeled:

```
UNTRUSTED_EXTERNAL_CONTENT
```

12. Document Extraction Upgrade

The following formats are supported:

```

.txt
.md
.pdf
.docx
.csv
.json

```

The pypdf package was added for PDF extraction.

The python-docx package was added for DOCX extraction.

The extraction implementation is in:

13. Natural-Language File Paths

The user can type a real path naturally:

Read C:\Users\Prajwal\Documents\project.pdf and summarize the methodology.

The application:

1. Extracts the path from the request.
2. Checks it against allowed roots.
3. Performs security preflight before reading.
4. Asks Ollama to propose read_file.
5. Sends the proposed action through the gateway.
6. Reads the file only after authorization.
7. Extracts and summarizes the document locally.

Default allowed roots include:

- Project directory.
- Sandbox directory.
- User Documents directory.
- User Desktop directory.

Path traversal is blocked before reading.

Example blocked path:

Read ../../Windows\System32\config\SAM and summarize it.

Result:

BLOCK
Path escapes the configured allowed file roots

14. Provenance and Capabilities

Actions now carry provenance and capability metadata.

Examples of provenance:

USER_INSTRUCTION
RAG_DOCUMENT
LOCAL_USER_DOCUMENT
AGENT_GENERATED

Examples of capabilities:

READ_FILE
WRITE_FILE
LOCAL_SEARCH
READ_DATABASE
EXTERNAL_NETWORK

The security engine evaluates capabilities rather than only tool names.

15. Risk and Policy Upgrade

Risk components now include:

- Injection risk.
- Provenance risk.
- Intent mismatch.
- Sensitive-data risk.
- Protected-path risk.
- Destination risk.
- Tool risk.
- TDG risk.
- Policy risk.

Risk weights and thresholds are configurable in:

config/policies.yaml

The thresholds are experimental project parameters, not universal security standards.

16. Real Attack Demonstration

The controlled malicious documents include:

vendor_malicious.txt
email_malicious.txt
README_malicious.md
malicious_policy.txt

Example malicious content asks the agent to access:

fake_secrets/api_key.txt

and transmit the result using an HTTP tool.

Protected flow:

User asks for summary
-> RAG retrieves malicious document
-> document is marked untrusted
-> Ollama proposes a dangerous action
-> gateway checks provenance
-> gateway checks intent mismatch
-> gateway checks external capability

-> policy blocks the action

Typical result:

Decision: BLOCK
Risk: 100 / 100
Source: RAG_DOCUMENT

The baseline mode bypasses the gateway for comparison. Its network operation is only an in-process mock and never contacts a real server.

17. Approval Workflow

High-risk operations such as local writes can return:

REQUIRE_APPROVAL

The dashboard shows an approval button. The action executes only after the user approves it.

Approval is logged separately in the audit trail.

18. Dashboard Evolution

The dashboard went through several stages.

Initial dashboard

The first dashboard showed basic counters, one request form, raw JSON, and recent audit events.

Control-center dashboard

Pages were added for:

- Live Monitor.
- Agent Console.
- Conversations.
- Documents.
- Threat Center.
- Attack Simulator.
- Tool Dependency Graph.
- RAG Explorer.
- Audit Explorer.
- Policy Center.
- Risk Analytics.
- Evaluation.
- System Health.

- Settings.
- Final Demo.

Dashboard repair

A `NameError` occurred because document was referenced before definition. The missing document selector was restored and the dashboard was verified with Python compilation and HTTP status 200.

Premium AI workspace redesign

The dashboard was then redesigned to make the AI chat the primary experience. It gained:

- Chat-style message bubbles.
- Local Ollama status.
- Theme selector.
- Conversation timestamps.
- New chat.
- Quick actions.
- Local document attachment selection.
- Live agent activity rail.
- Runtime security cards.
- Expandable analysis.
- Approval buttons.

Final navigation repair

The sidebar originally used multiple independent radio controls. This was replaced with one unified navigation selector containing all pages. The sidebar also received internal scrolling so the full navigation remains accessible.

The dashboard now presents backend-derived visual cards for threats, RAG, audit, health, risk, evaluation, and graph data instead of dumping raw JSON as the primary interface.

19. Dashboard Usage Examples

Normal question

Open Agent Console and enter:

What is the capital of France?

The agent answers without a tool.

Local file

Enter:

Read `C:\Users\Prajwal\llm-runtime-security\sandbox\documents\report.txt` and summarize it.

The chat shows a real `read_file` security card with an ALLOW decision.

RAG

Open RAG Explorer and query:

what is the company leave policy?

The page displays local chunks, similarity, source, and trust level.

Attack Lab

Open Attack Simulator, choose Malicious vendor document, choose PROTECTED, and run it.

The proposed dangerous action is shown with source, risk, signals, policy, and block result.

Audit

Open Audit Explorer and verify the SHA-256 audit chain.

20. Testing History

The project was tested incrementally.

Initial unit test result:

5 passed

After path and document upgrades:

6 passed

Compilation checks passed for the app, dashboard, scripts, and runner.

Live checks confirmed:

- Ollama health.
- Normal local chat.
- Local RAG.
- Natural absolute-path file reading.
- Path traversal blocking.
- Credential path blocking.
- Malicious RAG blocking.
- Dashboard HTTP 200.
- Document discovery endpoint.

- TDG endpoint.
- Policy endpoint.
- Audit verification.

21. Evaluation Results

The local evaluation used 50 benign and 50 malicious scenarios.

Measured results from the local Ollama benchmark included:

- Baseline attack success rate: 52%.
- Baseline unauthorized tool call rate: 52%.
- Protected detection/block rate: 26%.
- Protected false-positive rate: 0%.
- Protected false-negative rate for allowed malicious actions: 0%.
- Baseline average latency: approximately 6837 ms.
- Protected average latency: approximately 5843 ms.

The protected model sometimes refused to propose unsafe actions before the gateway received them. Those cases were kept as observed behavior and were not falsely counted as gateway detections.

22. GitHub History

Initial project commit:

1ce1381 Build runtime security gateway prototype

Ollama/V2 upgrade:

213a6be Upgrade platform to offline Ollama agent

Dashboard document-selector fix:

3104024 Fix dashboard document selector

V3 local-file and control-center upgrade:

2f52263 Add local file workflows and security control center

Premium dashboard redesign:

b8d57b9 Redesign dashboard as local AI security center

Final chat/sidebar/security-view repair:

9296d11 Repair dashboard navigation and security views

Latest chat workspace refinement:

23. Current Local Services

The final verified services are:

Ollama: 127.0.0.1:11434
FastAPI: 127.0.0.1:8000
Streamlit: 127.0.0.1:8501

24. Limitations

- The local model can refuse or misunderstand some tool tasks.
- IPI detection is not perfect.
- The dashboard uses polling and Streamlit reruns rather than a full WebSocket event bus.
- The vector store is a simple local SQLite implementation.
- The HTTP service is an in-process simulation.
- The audit hash chain is tamper-evident, not tamper-proof.
- The physical internet-disconnection test was not automated.
- No system can guarantee zero false positives or zero false negatives.

25. Recommended Learning Order

1. Python and virtual environments.
2. FastAPI request/response handling.
3. Streamlit state and reruns.
4. Ollama local model serving.
5. Structured LLM tool calls.
6. Pydantic validation.
7. Intent and provenance.
8. Risk and policy enforcement.
9. NetworkX dependency graphs.
10. Embeddings and semantic retrieval.
11. SQLite audit storage.
12. SHA-256 hash chains.
13. Dashboard investigation workflows.
14. Baseline versus protected evaluation.